

The Rabin-Karp algorithm

A different approach to exact matching

CMSC 423

String fingerprints give $O(n)$ expected-time exact matching



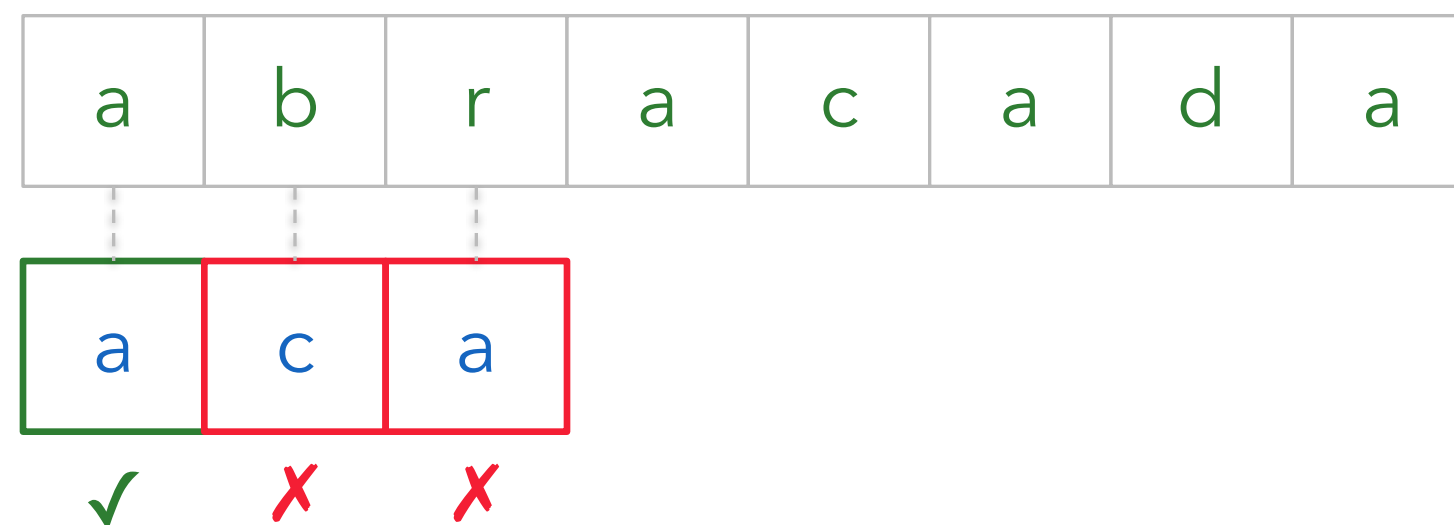
Eliminating spurious comparisons through “fingerprinting”

- Last time: the **Z-algorithm** – deterministic exact matching in $O(n + m)$ time.
- Today: a *probabilistic* approach. Instead of comparing substrings, compare short **fingerprints** – single numbers that stand in for a whole window of text.
- The key trick: the fingerprint of the next window can be derived from the current one in **$O(1)$** time. This is a **rolling hash**.
- Equal fingerprints do **not** prove equal strings, so every candidate match is verified character by character.

Why compare fingerprints instead of characters?

Naïve

compare every character of the window

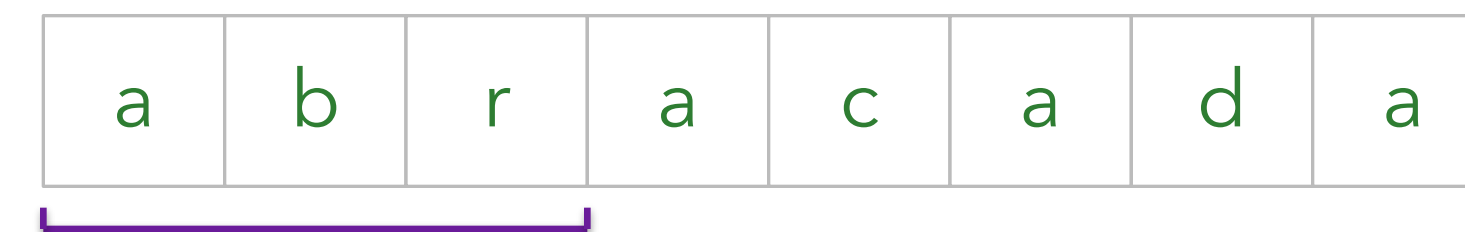


m comparisons, then shift by one

$O(mn)$ worst case

Rabin-Karp

compare one number for the window



$$t_s = 4 \quad \text{vs} \quad p = 9$$

$4 \neq 9 \rightarrow$ no match here ✗

1 comparison, then shift by one

$O(n)$ expected

Rabin-Karp trades m character comparisons per shift for a single number comparison – and computes the next number in **$O(1)$** time.

Characters as digits

- Assume $\Sigma = \{0, 1, \dots, 9\}$. Then any string is just a decimal number.
- Write $|P| = m$ and $|T| = n$. Let p be the number denoted by P , and t_s the number denoted by the window $T[s .. s+m-1]$.

$$p = P[0] \cdot 10^{m-1} + P[1] \cdot 10^{m-2} + \dots + P[m-2] \cdot 10 + P[m-1]$$

$$P \text{ occurs at position } s \text{ of } T \iff p = t_s$$

- For a general alphabet of size d , replace 10 by d everywhere. (DNA: $d = 4$.)
- Catch: p has m digits, so it grows exponentially in m – far too big to hold in a machine word. The next two slides fix this.

If the pattern is “small”, comparison is $O(1)$

- If p and t_s both fit in a single machine word, then testing $p = t_s$ is one instruction – independent of m .
- Two things still need solving:
 - 1 How do we compute the very first fingerprint t_0 without ever forming a huge number?
 - 2 How do we get t_{s+1} from t_s without re-reading the whole window?

Preview – DNA. With $\Sigma = \{A, C, G, T\}$ we need only 2 bits per base, so $64 / 2 = 32$ bases fit in one 64-bit word.

For patterns that short, no modular arithmetic is needed at all – and there are no false positives.

Computing p with Horner's rule

Evaluating the polynomial term by term costs $O(m)$ multiplications and builds enormous intermediate values. Horner's rule nests them instead:

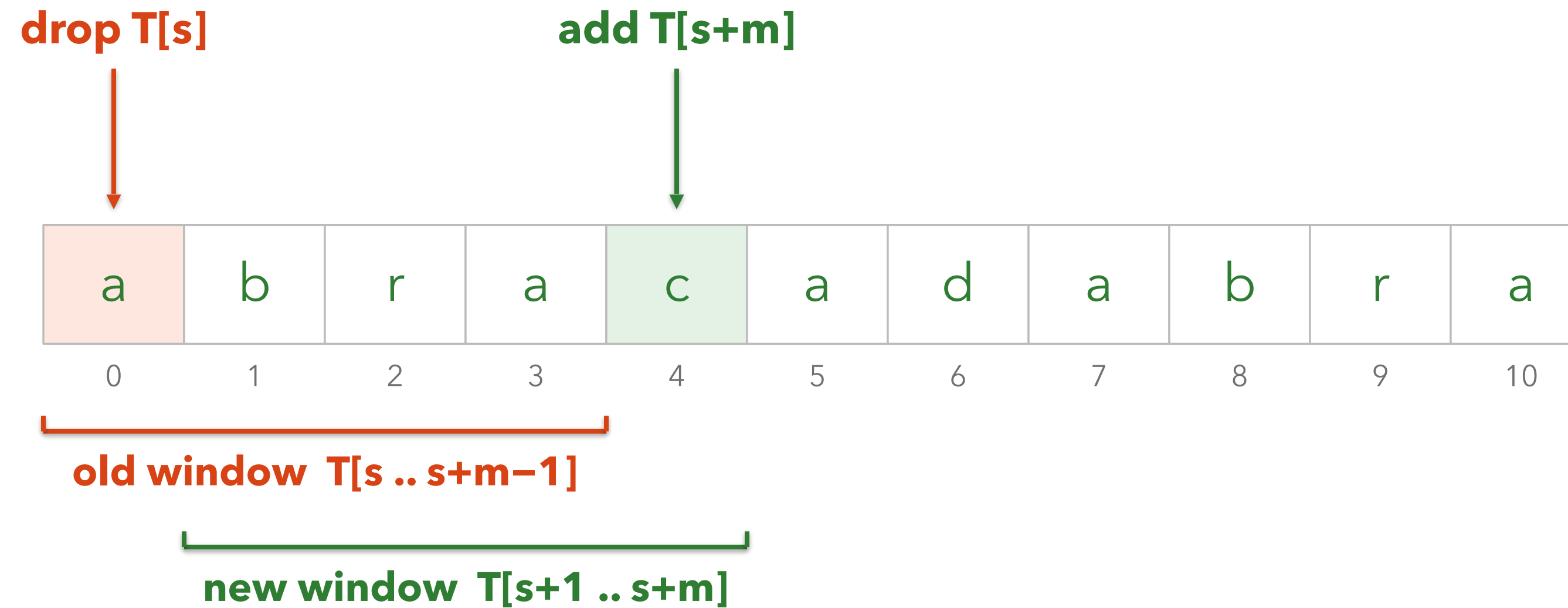
$$p = P[m-1] + 10 \cdot (P[m-2] + 10 \cdot (P[m-3] + \cdots + 10 \cdot P[0] \cdots))$$

– that is, $p \leftarrow (d \cdot p + P[i])$ repeated for $i = 0 \dots m-1$.

- One multiply and one add per character: $O(m)$ time, and *with a $(\text{mod } q)$ on each step the running value never exceeds q .* **Note:** This works because
$$(A+B) \bmod q = ((A \bmod q) + (B \bmod q)) \bmod q$$
$$(A \cdot B) \bmod q = ((A \bmod q) \cdot (B \bmod q)) \bmod q$$
- The same loop over $T[0 \dots m-1]$ produces t_0 .

Precompute once: $h = d^{m-1} \bmod q$ – the weight of the leading character

The rolling hash: one window to the next



$$t_{s+1} = (d \cdot (t_s - h \cdot T[s]) + T[s+m]) \bmod q$$

Subtract the leading character (scaled by $h = d^{m-1}$), shift everything left by multiplying by d , then add the incoming character. Constant work per shift.

Rabin-Karp: the algorithm

Problem. Fingerprints get astronomically large: $256^{10} \approx 1.2 \times 10^{24}$ for $m = 10, d = 256$.

Solution. Work modulo a prime q , so every value stays below q and fits in a word.

```
h ← dm-1 mod q
p ← 0;    t ← 0
for i = 0 ... m-1:
    p ← (d·p + P[i]) mod q
    t ← (d·t + T[i]) mod q
for s = 0 ... n-m:
    if p = t and P = T[s .. s+m-1]:    report a match at s
    t ← (d·(t - h·T[s]) + T[s+m]) mod q
```

New problem. Now $p = t_s$ now only means the fingerprints agree – the strings may still differ, hence the explicit check.

False positives: a concrete example

$T = \text{"153826"}$ $P = \text{"26"}$ $d = 10$ $q = 11$ $p = 26 \bmod 11 = 4$

s	$T[s .. s+1]$	t_s	$t_s = p ?$	actually "26" ?
0	"15"	4	yes	NO ✗
1	"53"	9	no	—
2	"38"	5	no	—
3	"82"	5	no	—
4	"26"	4	yes	YES ✓

$s = 0$: "15" and "26" have the same fingerprint (both $\equiv 4 \bmod 11$) but are different strings.

A fingerprint match is only a **candidate**. Always verify the m characters before reporting a match.

Worked example: setup

T = “try eduroam; it won't work” P = “eduroam”

t	r	y	_	e	d	u	r	o	a	m	;	_	i	t	_	w	o	n	'	t	_	w	o	r	k
0	2			4	5	6	7	8	9	10		12	13	14	15	16	17	18	19	20	21	22	23	24	

the only occurrence starts at $s = 4$

- $n = 26$, $m = 7$, $d = 256$ (one byte per character), $q = 101$
- $h = d_{m-1} \bmod q = 256^6 \bmod 101 = \mathbf{25}$
- Horner over P gives $p = \mathbf{72}$. We now roll a length- m window across T and watch for $t_s = \mathbf{72}$.

Worked example: rolling the window

$T = \text{"try eduroam; it won't work"}$ $P = \text{"eduroam"}$ $d = 256$ $q = 101$ $h = 25$ $p = 72$

s	t_s	drop $T[s]$	add $T[s+m]$	t_{s+1}
0	2	't' (116)	'r' (114)	71
1	71	'r' (114)	'o' (111)	30
2	30	'y' (121)	'a' (97)	68
3	68	'_' (32)	'm' (109)	72
4	72	'e' (101)	',' (59)	8

$$t_{s+1} = (256 \cdot (t_s - 25 \cdot T[s]) + T[s+m]) \bmod 101$$

Each row costs one subtract, one multiply, one add and one remainder – no matter how long the pattern is.

Worked example: every fingerprint

$T = \text{"try eduroam; it won't work"}$ $P = \text{"eduroam"}$ $d = 256$ $q = 101$ $h = 25$ $p = 72$

s	0	1	2	3	4	5	6	7	8	9
t_s	2	71	30	68	72	8	97	4	53	100

s	10	11	12	13	14	15	16	17	18	19
t_s	11	5	15	69	58	84	37	29	98	16

$t_4 = 72 = p \rightarrow \text{verify } T[4 \dots 10] = \text{"eduroam"} \checkmark$

Exactly one fingerprint collision occurs here, and it is the true match – no wasted verification.

Where Rabin-Karp sits

Algorithm	Preprocessing	Search	Notes
Naïve	$O(1)$	$O(mn)$ worst	no preprocessing
Z-algorithm	$O(n + m)$	$O(n + m)$ always	exact, deterministic
Rabin-Karp	$O(m)$	$O(n)$ expected, $O(mn)$ worst	probabilistic; fast in practice

Why $O(n)$ expected? With a well-chosen prime q , a spurious fingerprint collision happens with probability about $1/q$ at each position. Across n positions that is $O(n/q)$ expected verifications, each costing $O(m)$. For $q \geq m$ the total is $O(n)$.

The $O(mn)$ worst case needs an adversarial text that collides at every position – vanishingly unlikely if q is chosen at random from a large set of primes.

Rabin-Karp notes

- **Short patterns.** If the whole pattern fits in a machine word, drop the $(\text{mod } q)$ entirely. Fingerprints are then exact and **no false positives are possible**.
- **Several primes.** Keeping fingerprints modulo two or three different primes makes a simultaneous collision far less likely, at negligible extra cost.
- **DNA.** $|\Sigma| = 4$, so 2 bits per nucleotide, and $64 \div 2 = \mathbf{32 \text{ nucleotides}}$ fit exactly in one 64-bit word – long enough for most seed lookups in read mapping.

Further reading

ntHash: recursive nucleotide hashing – Mohamadi, Chu, Vandervalk & Birol, *Bioinformatics* 32(22), 2016. A rolling hash designed specifically for DNA k-mers.

Rabin-Karp in code

```
def rabin_karp(T, P, d=256, q=101):
    n, m = len(T), len(P)
    h = pow(d, m - 1, q)          # weight of the leading char
    p = t = 0
    for i in range(m):           # Horner: O(m) preprocessing
        p = (d * p + ord(P[i])) % q
        t = (d * t + ord(T[i])) % q
    for s in range(n - m + 1):
        if p == t and T[s:s+m] == P:  # verify!
            yield s
        if s < n - m:                 # roll in O(1)
            t = (d * (t - ord(T[s]) * h)
                  + ord(T[s+m])) % q
```

1 Precompute $h = d^{m-1} \bmod q$ – the weight of the character that leaves the window.

2 Horner's rule builds p and t_0 in $O(m)$ without ever forming a huge number.

3 The rolling update: drop, shift, add – all mod q , in $O(1)$.